



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

Ayudantía 01: C# parte 2

Francisco Gazitúa Requena (fco_gazitua_requena@uc.cl)

IIC2113 Diseño Detallado de Software

8 de Agosto, 2025

Consola

La clase `Console` dentro del namespace `System` nos permite interactuar con la consola.

La clase `Console` dentro del namespace `System` nos permite interactuar con la consola.

Escribir:

- `Console.Write(..)`: Muestra el texto en consola.
- `Console.WriteLine(..)`: Muestra el texto en consola y agrega un salto de línea.

Leer:

- `Console.ReadLine()`: Retorna un `string` con la línea escrita en consola.

La clase Console dentro del namespace System nos permite interactuar con la consola.

Escribir:

- `Console.Write(..)`: Muestra el texto en consola.
- `Console.WriteLine(..)`: Muestra el texto en consola y agrega un salto de línea.

Leer:

- `Console.ReadLine()`: Retorna un string con la línea escrita en consola.

Para convertir un string en otro tipo de dato se usa la clase Convert.

```
2 string str = Console.ReadLine();  
3 int intVar = Convert.ToInt32(str);  
4 double doubleVar = Convert.ToDouble(str);
```

Listas y Diccionarios

El namespace `System.Collections` incluye las clases `List` y `Dictionary`. En ambos casos se **debe definir el tipo de dato** sobre el que operan.

Listas

El namespace `System.Collections` incluye las clases `List` y `Dictionary`. En ambos casos se **debe definir el tipo de dato** sobre el que operan.

`List<tipo>` funciona prácticamente igual que una lista en python, pero solo puede contener datos del mismo tipo (que se define al crear la lista).

```
1 // Creamos una lista que solo puede contener strings
2 List<string> listaDeStrings = new List<string>();
3
4
5
6
7
8
9
10
11
```

Listas

El namespace `System.Collections` incluye las clases `List` y `Dictionary`. En ambos casos se **debe definir el tipo de dato** sobre el que operan.

`List<tipo>` funciona prácticamente igual que una lista en python, pero solo puede contener datos del mismo tipo (que se define al crear la lista).

```
1 // Creamos una lista que solo puede contener strings
2 List<string> listaDeStrings = new List<string>();
3 listaDeStrings.Add("hola");
4 listaDeStrings.Add("auto");
5 listaDeStrings.Add("mañana");
6
7
8
9
10
11
```

Listas

El namespace `System.Collections` incluye las clases `List` y `Dictionary`. En ambos casos se **debe definir el tipo de dato** sobre el que operan.

`List<tipo>` funciona prácticamente igual que una lista en python, pero solo puede contener datos del mismo tipo (que se define al crear la lista).

```
1 // Creamos una lista que solo puede contener strings
2 List<string> listaDeStrings = new List<string>();
3 listaDeStrings.Add("hola");
4 listaDeStrings.Add("auto");
5 listaDeStrings.Add("mañana");
6 Console.WriteLine(listaDeStrings[1]); // muestra "auto"
7 Console.WriteLine(listaDeStrings.Count); // largo de la lista
8 listaDeStrings.Remove("auto");
9 Console.WriteLine(listaDeStrings[1]); // muestra "mañana"
10
11
```

Listas

El namespace `System.Collections` incluye las clases `List` y `Dictionary`. En ambos casos se **debe definir el tipo de dato** sobre el que operan.

`List<tipo>` funciona prácticamente igual que una lista en python, pero solo puede contener datos del mismo tipo (que se define al crear la lista).

```
1 // Creamos una lista que solo puede contener strings
2 List<string> listaDeStrings = new List<string>();
3 listaDeStrings.Add("hola");
4 listaDeStrings.Add("auto");
5 listaDeStrings.Add("mañana");
6 Console.WriteLine(listaDeStrings[1]); // muestra "auto"
7 Console.WriteLine(listaDeStrings.Count); // largo de la lista
8 listaDeStrings.Remove("auto");
9 Console.WriteLine(listaDeStrings[1]); // muestra "mañana"
10 // Pero NO puedes agregar elementos que no sean del tipo de la lista
11 listaDeStrings.Add(3.2); // -> ERROR!
```

Para recorrer una lista podemos usar un for

```
2 List<int> listaDeInts = new List<int>();  
3 listaDeInts.Add(93);  
4 listaDeInts.Add(2);  
5 listaDeInts.Add(4);  
6 for (int i = 0; i < listaDeInts.Count; i++)  
7 {  
8     int num = listaDeInts[i];  
9     Console.WriteLine($"Este numero es {num}");  
10 }
```

... o un foreach:

```
2 List<int> listaDeInts = new List<int>();  
3 listaDeInts.Add(93);  
4 listaDeInts.Add(2);  
5 listaDeInts.Add(4);  
6 foreach(int num in listaDeInts)  
7     Console.WriteLine($"Este numero es {num}");
```

Los `Dictionary<tipo llave, tipo elemento>` también funcionan prácticamente igual que en python, pero tienen tipos fijos de llaves y elementos.

Los Dictionary<tipo llave, tipo elemento> también funcionan prácticamente igual que en python, pero tienen tipos fijos de llaves y elementos.

```
1 // En este caso, las llaves son string y los elementos son ints
2 Dictionary<string, int> dict = new Dictionary<string, int>();
3
4
5
6
7
8
9
10
11
12
13
14
```

Los Dictionary<tipo llave, tipo elemento> también funcionan prácticamente igual que en python, pero tienen tipos fijos de llaves y elementos.

```
1 // En este caso, las llaves son string y los elementos son ints
2 Dictionary<string, int> dict = new Dictionary<string, int>();
3 dict["siete"] = 7;
4 dict["cuatro"] = 4;
```

Los Dictionary<tipo llave, tipo elemento> también funcionan prácticamente igual que en python, pero tienen tipos fijos de llaves y elementos.

```
1 // En este caso, las llaves son string y los elementos son ints
2 Dictionary<string, int> dict = new Dictionary<string, int>();
3 dict["siete"] = 7;
4 dict["cuatro"] = 4;
5 Console.WriteLine(dict["cuatro"]); // Muestra 4
6
7
8
9
10
11
12
13
14
```

Los Dictionary<tipo llave, tipo elemento> también funcionan prácticamente igual que en python, pero tienen tipos fijos de llaves y elementos.

```
1 // En este caso, las llaves son string y los elementos son ints
2 Dictionary<string, int> dict = new Dictionary<string, int>();
3 dict["siete"] = 7;
4 dict["cuatro"] = 4;
5 Console.WriteLine(dict["cuatro"]); // Muestra 4
6 // Recorrer las llaves
7 foreach (string llave in dict.Keys)
8     Console.WriteLine(llave);
```

Los Dictionary<tipo llave, tipo elemento> también funcionan prácticamente igual que en python, pero tienen tipos fijos de llaves y elementos.

```
1 // En este caso, las llaves son string y los elementos son ints
2 Dictionary<string, int> dict = new Dictionary<string, int>();
3 dict["siete"] = 7;
4 dict["cuatro"] = 4;
5 Console.WriteLine(dict["cuatro"]); // Muestra 4
6 // Recorrer las llaves
7 foreach (string llave in dict.Keys)
8     Console.WriteLine(llave);
9 // Recorrer los valores
10 foreach (int valor in dict.Values)
11     Console.WriteLine(valor);
12
13
14
```

Los Dictionary<tipo llave, tipo elemento> también funcionan prácticamente igual que en python, pero tienen tipos fijos de llaves y elementos.

```
1 // En este caso, las llaves son string y los elementos son ints
2 Dictionary<string, int> dict = new Dictionary<string, int>();
3 dict["siete"] = 7;
4 dict["cuatro"] = 4;
5 Console.WriteLine(dict["cuatro"]); // Muestra 4
6 // Recorrer las llaves
7 foreach (string llave in dict.Keys)
8     Console.WriteLine(llave);
9 // Recorrer los valores
10 foreach (int valor in dict.Values)
11     Console.WriteLine(valor);
12 // Recorrer ambos
13 foreach (var e in dict)
14     Console.WriteLine($"{e.Key} -> {e.Value}");
```

Manejo de Archivos

El namespace `System.IO` contiene todo lo que podrías necesitar para manejar archivos.

Manejo de Archivos

El namespace `System.IO` contiene todo lo que podrías necesitar para manejar archivos.

La carpeta donde inicia tu programa es donde se encuentra el binario que se genera al compilar el código $\approx$ `bin/Debug/net8.0/`. Digamos que contiene lo siguiente:

- `pictures/Lenna.png`
- `imgs`

Manejo de Archivos

El namespace `System.IO` contiene todo lo que podrías necesitar para manejar archivos.

La carpeta donde inicia tu programa es donde se encuentra el binario que se genera al compilar el código $\approx$ `bin/Debug/net8.0/`. Digamos que contiene lo siguiente:

- `pictures/Lenna.png`
- `imgs`

Ejemplo:

```
2 string file = "pictures/Lenna.png";
3 if (File.Exists(file))
4 {
5     File.Copy(file, "imgs/Lenna.png");
6     File.Delete(file);
7     File.Move("imgs/Lenna.png", file);
8 }
```

¿Qué hace el siguiente código?

Manejo de Archivos

El namespace `System.IO` contiene todo lo que podrías necesitar para manejar archivos.

La carpeta donde inicia tu programa es donde se encuentra el binario que se genera al compilar el código $\approx$ `bin/Debug/net8.0/`. Digamos que contiene lo siguiente:

- `pictures/Lenna.png`
- `imgs`

Ejemplo:

```
2 string file = "pictures/Lenna.png";
3 if (File.Exists(file))
4 {
5     File.Copy(file, "imgs/Lenna.png");
6     File.Delete(file);
7     File.Move("imgs/Lenna.png", file);
8 }
```

¡Este código no funciona en Windows!

Las rutas en Mac y Linux se separan con /:

```
2 string file = "pictures/Lenna.png";
```

Manejo de Archivos

Las rutas en Mac y Linux se separan con /:

```
2 string file = "pictures/Lenna.png";
```

Pero las rutas en Windows se separan con \.

```
1 string file = "pictures\\Lenna.png";  
2 if (File.Exists(file))  
3 {  
4     File.Copy(file, "imgs\\Lenna.png");  
5     File.Delete(file);  
6     File.Move("imgs\\Lenna.png", file);  
7 }
```

Para que el código funcione en cualquier SO hay que usar `Path.Combine(...)`

```
1 string file = Path.Combine("pictures", "Lenna.png");
2 if (File.Exists(file))
3 {
4     File.Copy(file, Path.Combine("imgs", "Lenna.png"));
5     File.Delete(file);
6     File.Move(Path.Combine("imgs", "Lenna.png"), file);
7 }
```

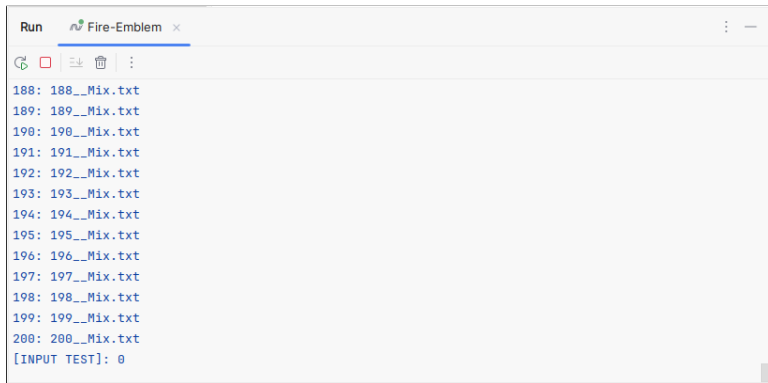
... este es un error típico del proyecto que mata los test cases :/

Manejo de Archivos

Directory: Permite manejar carpetas, crearlas, obtener los archivos en ella, etc.

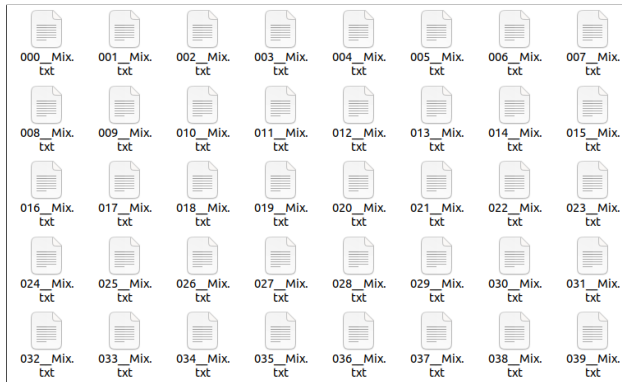
```
1 // Creamos una carpeta si no existe
2 string folder = "imgs";
3 if(!Directory.Exists(folder))
4     Directory.CreateDirectory(folder);
5
6 // Leemos los archivos .png en una carpeta y los mostramos en consola
7 string[] files = Directory.GetFiles("pictures", "*.png");
8 foreach (string file in files)
9     Console.WriteLine(file);
```

Otro error típico del proyecto.



```
Run Fire-Emblem x
188: 188__Mix.txt
189: 189__Mix.txt
190: 190__Mix.txt
191: 191__Mix.txt
192: 192__Mix.txt
193: 193__Mix.txt
194: 194__Mix.txt
195: 195__Mix.txt
196: 196__Mix.txt
197: 197__Mix.txt
198: 198__Mix.txt
199: 199__Mix.txt
200: 200__Mix.txt
[INPUT TEST]: 0
```

Otro error típico del proyecto.



Otro error típico del proyecto.

```
1 view.WriteLine("Elige un archivo para cargar los equipos");
2 string[] teamFiles = Directory.GetFiles(teamsFolder, "*.txt");
3 for(int i = 0; i < teamFiles.Length; i++)
4 {
5     string teamFile = Path.GetFileName(options[i]);
6     view.WriteLine($"{i}: {teamFile}");
7 }
```

Otro error típico del proyecto.



```
Run  Fire-Emblem x
189: 012__Mix.txt
190: 132__Mix.txt
191: 166__Mix.txt
192: 058__Mix.txt
193: 162__Mix.txt
194: 084__Mix.txt
195: 083__Mix.txt
196: 153__Mix.txt
197: 140__Mix.txt
198: 063__Mix.txt
199: 117__Mix.txt
200: 149__Mix.txt
[ERROR] el valor esperado acá era: "0: 000__Mix.txt"
[INPUT MANUAL]:
```

Otro error típico del proyecto.

```
1 view.WriteLine("Elige un archivo para cargar los equipos");
2 string[] teamFiles = Directory.GetFiles(teamsFolder, "*.txt");
3 Array.Sort(teamFiles);
4 for(int i = 0; i < teamFiles.Length; i++)
5 {
6     string teamFile = Path.GetFileName(options[i]);
7     view.WriteLine($"{i}: {teamFile}");
8 }
```

Otro error típico del proyecto.

```
1 view.WriteLine("Elige un archivo para cargar los equipos");
2 string[] teamFiles = Directory.GetFiles(teamsFolder, "*.txt");
3 Array.Sort(teamFiles);
4 for(int i = 0; i < teamFiles.Length; i++)
5 {
6     string teamFile = Path.GetFileName(options[i]);
7     view.WriteLine($"{i}: {teamFile}");
8 }
```

Ahora el código funciona en cualquier S.O. :)

Lectura y Escritura de Archivos

Podemos leer y escribir archivos de texto usando `File`.

Lectura y Escritura de Archivos

Podemos leer y escribir archivos de texto usando File.

Escribir un archivo:

```
1 string fileName = "myFile.txt";  
2 string[] lines = new []{"line 1", "line 2", "line 3"};  
3 File.WriteAllLines(fileName, lines);
```

El archivo queda guardado en la carpeta bin/Debug/net8.0 (o equivalente)

Lectura y Escritura de Archivos

Podemos leer y escribir archivos de texto usando File.

Escribir un archivo:

```
1 string fileName = "myFile.txt";  
2 string[] lines = new []{"line 1", "line 2", "line 3"};  
3 File.WriteAllLines(fileName, lines);
```

El archivo queda guardado en la carpeta bin/Debug/net8.0 (o equivalente)

Leer un archivo:

```
1 string fileName = "myFile.txt";  
2 string[] lines = File.ReadAllLines(fileName);
```

En este caso, leemos el archivo ubicado en bin/Debug/net8.0/myFile.txt

Serialización en JSON

Serialización

Digamos que te dan un JSON y te piden que lo importes en C#:

```
1  [  
2    {  
3      "Name": "Alexis",  
4      "LastName": "Sanchez",  
5      "Age": 33  
6    },  
7    {  
8      "Name": "Arturo",  
9      "LastName": "Vidal",  
10     "Age": 35  
11   }  
12 ]
```

Hay muchas formas de hacer esto (ver: learn.microsoft.com), pero acá les mostraré la forma más simple de todas.

Ir a <https://json2csharp.com/>

Convert Json to C# Classes Online

Convert any JSON object to a C# class online. Check out the help panel below to view details on how to use this converter.

The screenshot shows the web interface of the 'Convert Json to C# Classes Online' tool. At the top, there is a navigation bar with icons for various conversion paths: JSON to C#, XML to C#, C# to JSON, C# to XML, JSON to Java, XML to Java, Java to JSON, Java to XML, JSON to Python, Python to JSON, JSON to Dart, Dart to JSON, and Swift to JSON. Below the navigation bar is a large text area for input and output, with line numbers 1 visible on both sides. In the center of the text area, there are two large icons: a yellow icon with curly braces and a vertical bar labeled 'JSON' on the left, and a blue icon with a 'C#' symbol on the right. At the bottom left, there is a 'Property Settings' panel with checkboxes for 'Use Pascal Case' and 'Use Fields'. At the bottom center, there is a green 'Convert' button with a lightning bolt icon. At the bottom right, there is a dark grey 'Copy To Clipboard' button with a clipboard icon, and below it, a 'Namespace' input field and a 'Zip as Files' button with a download icon.

Ir a <https://json2csharp.com/>

Convert Json to C# Classes Online

Convert any JSON object to a C# class online. Check out the help panel below to view details on how to use this converter.



```
1 {  
2   {  
3     "Name": "Alexis",  
4     "LastName": "Sanchez",  
5     "Age": 33  
6   },  
7   {  
8     "Name": "Arturo",  
9     "LastName": "Vidal",  
10    "Age": 35  
11  }  
12 }
```



Property Settings

☐ Use Pascal Case

☐ Use Fields

Convert

Copy To Clipboard

Namespace

Zip as Files

Ir a <https://json2csharp.com/>

Convert Json to C# Classes Online
Convert any JSON object to a C# class online. Check out the help panel below to view details on how to use this converter.

JSON → C# XML → C# JSON → C# YAML → C# TOML → C# CSV → C# Dart Swift → Kotlin

```
1 [
2   {
3     "Name": "Alexis",
4     "LastName": "Sanchez",
5     "Age": 33
6   },
7   {
8     "Name": "Arturo",
9     "LastName": "Vidal",
10    "Age": 35
11  }
12 ]
```

```
1 // Root myDeserializedClass = JsonConvert.DeserializeObject<List<
2   public class Root
3   {
4     public string Name { get; set; }
5     public string LastName { get; set; }
6     public int Age { get; set; }
7   }
8
9
```

Property Settings

- ☐ Use Pascal Case
- ☐ Use Fields

Convert ⚡

Generate Examples ⚡

Copy To Clipboard 📄

Namespace

Zip as Files 📦

Serialización

La clase generada es una clase auxiliar cuya única responsabilidad es guardar la información del JSON leído (de hecho tiene atributos públicos por lo mismo).

```
1 public class PlayerInfo
2 {
3     public string Name { get; set; }
4     public string LastName { get; set; }
5     public int Age { get; set; }
6 }
```

Serialización

La clase generada es una clase auxiliar cuya única responsabilidad es guardar la información del JSON leído (de hecho tiene atributos públicos por lo mismo).

```
1 public class PlayerInfo
2 {
3     public string Name { get; set; }
4     public string LastName { get; set; }
5     public int Age { get; set; }
6 }
```

Para leer el JSON usamos el namespace: `System.Text.Json`

```
1 using System.Text.Json;
2
3 string myJson = File.ReadAllText("data.json");
4 var players = JsonSerializer.Deserialize<List<PlayerInfo>>(myJson);
5 foreach(var player in players)
6     Console.WriteLine($"{player.Name} {player.LastName}: {player.Age}");
```

Serialización

La clase generada es una clase auxiliar cuya única responsabilidad es guardar la información del JSON leído (de hecho tiene atributos públicos por lo mismo).

```
1 public class PlayerInfo
2 {
3     public string Name { get; set; }
4     public string LastName { get; set; }
5     public int Age { get; set; }
6 }
```

Guardar un JSON es igual de fácil.

```
1 using System.Text.Json;
2
3 string fileName = "MyData.json";
4 string jsonString = JsonSerializer.Serialize(players);
5 File.WriteAllText(fileName, jsonString);
```

Modificadores Adicionales

Modificadores adicionales: static

Normalmente, para usar los métodos de una clase tenemos que crear un objeto de ella.

```
2 Random rnd = new Random();  
3 int rndInt = rnd.Next();
```

Modificadores adicionales: static

Normalmente, para usar los métodos de una clase tenemos que crear un objeto de ella.

```
2 Random rnd = new Random();  
3 int rndInt = rnd.Next();
```

Pero Math y Console nos dejan llamar a sus métodos sin crear un objeto.

```
2 double num = Math.Pow(2, 5);  
3 Console.WriteLine(num);
```

¿Cómo lo hacen? 🤖

Modificadores adicionales: static

Normalmente, para usar los métodos de una clase tenemos que crear un objeto de ella.

```
2 Random rnd = new Random();  
3 int rndInt = rnd.Next();
```

Pero Math y Console nos dejan llamar a sus métodos sin crear un objeto.

```
2 double num = Math.Pow(2, 5);  
3 Console.WriteLine(num);
```

¿Cómo lo hacen? 🤖

C# nos permite definir métodos y atributos que están asociados a la clase misma y no a sus objetos.

Modificadores adicionales: static

Normalmente, para usar los métodos de una clase tenemos que crear un objeto de ella.

```
2 Random rnd = new Random();  
3 int rndInt = rnd.Next();
```

Pero Math y Console nos dejan llamar a sus métodos sin crear un objeto.

```
2 double num = Math.Pow(2, 5);  
3 Console.WriteLine(num);
```

¿Cómo lo hacen? 🤖

C# nos permite definir métodos y atributos que están asociados a la clase misma y no a sus objetos. ¿Qué significa esto?

Modificadores adicionales: `static`

Partamos por los atributos estáticos.

Modificadores adicionales: `static`

Partamos por los atributos estáticos. Si definimos un atributo como `static` entonces ese atributo tendrá el mismo valor para **todos** los objetos de la clase.

Modificadores adicionales: static

Partamos por los atributos estáticos. Si definimos un atributo como `static` entonces ese atributo tendrá el mismo valor para **todos** los objetos de la clase.

```
2 public class A {  
3     public static int StcAtt = 0;  
4     public void AddOne() { StcAtt++; }  
5     public void Print() {  
6         Console.WriteLine("El atributo estático es " + StcAtt);  
7     }  
8 }
```

Modificadores adicionales: static

Partamos por los atributos estáticos. Si definimos un atributo como `static` entonces ese atributo tendrá el mismo valor para **todos** los objetos de la clase.

```
2 public class A {  
3     public static int StcAtt = 0;  
4     public void AddOne() { StcAtt++; }  
5     public void Print() {  
6         Console.WriteLine("El atributo estático es " + StcAtt);  
7     }  
8 }
```

```
2 Console.WriteLine(A.StcAtt); // 0  
3 A.StcAtt++;  
4 Console.WriteLine(A.StcAtt); // 1  
5 A a = new A();  
6 a.AddOne(); // Sumamos 1 al valor estático desde objeto 'a'  
7 A b = new A();  
8 b.Print(); // "El atributo estático es 2"
```

Modificadores adicionales: static

También podemos definir métodos estáticos dentro de clases.

```
2 public class A {  
3     private static int _stcAtt = 0;  
4     public static void AddOne() { _stcAtt++; }  
5     public static void Print() {  
6         Console.WriteLine("El atributo estático es " + _stcAtt);  
7     }  
8 }
```

Modificadores adicionales: static

También podemos definir métodos estáticos dentro de clases.

```
2 public class A {  
3     private static int _stcAtt = 0;  
4     public static void AddOne() { _stcAtt++; }  
5     public static void Print() {  
6         Console.WriteLine("El atributo estático es " + _stcAtt);  
7     }  
8 }
```

```
2 A.Print(); // El atributo estático es 0  
3 A.AddOne();  
4 A.Print(); // El atributo estático es 1
```

Modificadores adicionales: static

También podemos definir métodos estáticos dentro de clases.

```
2 public class A {  
3     private static int _stcAtt = 0;  
4     public static void AddOne() { _stcAtt++; }  
5     public static void Print() {  
6         Console.WriteLine("El atributo estático es " + _stcAtt);  
7     }  
8 }
```

```
2 A.Print(); // El atributo estático es 0  
3 A.AddOne();  
4 A.Print(); // El atributo estático es 1
```

Observaciones:

- Los métodos estáticos solo pueden acceder a variables y métodos estáticos.
- Pero un método **no** estático puede acceder sin problemas a todos los elementos estáticos de la clase.

Modificadores adicionales: static

Importante: No usen atributos estáticos en el proyecto.

Modificadores adicionales: static

Importante: No usen atributos estáticos en el proyecto.

[E2] Consola muestra resultados correctos, pero el test falla. #41

 Closed

TFreyre23 opened this issue on Apr 21 · 6 comments

Output esperado calza pero lo considera "Failed" #172

 Closed

paulmacguire opened this issue on Sep 20, 2023 · 7 comments

Unit Test Error, pero el error no existe #106

 Open

felipecuentasporras opened this issue on Jun 11, 2023 · 3 comments

... entre otros.

Modificadores adicionales: `const`

Otros modificadores que resultan útiles son `const` y `readonly`. Ambos limitan los cambios que se puede hacer sobre el valor de un atributo o variable.

Modificadores adicionales: const

Otros modificadores que resultan útiles son `const` y `readonly`. Ambos limitan los cambios que se puede hacer sobre el valor de un atributo o variable.

`const`

Indica que el valor de la variable/atributo es **constante** durante toda la ejecución del programa. Este valor debe ser asignado al momento de declarar la variable. Además, la variable debe ser un tipo por valor, un `string` o `null`.

```
2  const double pi = 3.1415;  
3  // podemos usar la variable  
4  double r = 1;  
5  double area = pi*r*r;  
6  // pero no podemos cambiar su valor  
7  pi = 3.141592; // ERROR!
```

Modificadores adicionales: const

Otros modificadores que resultan útiles son `const` y `readonly`. Ambos limitan los cambios que se puede hacer sobre el valor de un atributo o variable.

`readonly`

Indica que el valor de la atributo **no puede cambiar una vez inicializado**. En este caso, el atributo puede ser declarado y darle un valor más adelante. Pero una vez que dicho valor es asignado no se podrá cambiar.

```
2 class Person {
3     private static int _rutActual = 0;
4     // definimos el rut como readonly porque no queremos
5     // permitir que se cambie el rut de las personas.
6     private readonly int _rut;
7     public Person() {
8         _rutActual++;
9         _rut = _rutActual; }
10 }
```

Punteros en C#

Punteros en C#

C# esconde el manejo de punteros mediante definir dos tipos de datos:

C# esconde el manejo de punteros mediante definir dos tipos de datos:

- **Tipos por valor:**

- Tipos básicos: `int`, `float`, `double`, `short`, ...
- Estructuras: `struct`.
- Enumeraciones: `enum`.

- **Tipos por referencia:**

- Clases (predefinidas y creadas por nosotros).
- Arreglos.
- strings.

C# esconde el manejo de punteros mediante definir dos tipos de datos:

- **Tipos por valor:**

- Tipos básicos: `int`, `float`, `double`, `short`, ...
- Estructuras: `struct`.
- Enumeraciones: `enum`.

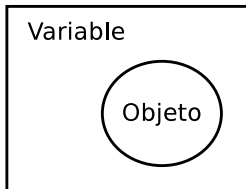
- **Tipos por referencia:**

- Clases (predefinidas y creadas por nosotros).
- Arreglos.
- strings.

La diferencia radica en el contenido de la variable declarada!

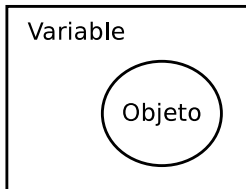
- **Value:** La variable contiene el valor mismo del objeto.
- **Reference:** La variable contiene una referencia al objeto (i.e., un puntero).

Por valor: La variable *contiene* al objeto.



Ejemplos: `int`, `struct`, `enum` y `Tuples`.

Por valor: La variable *contiene* al objeto.



Ejemplos: `int`, `struct`, `enum` y `Tuples`.

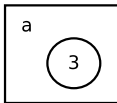
Consecuencia: `b = a`, copia objeto `a` en `b`.

Ejemplo 1:

```
2 int a = 3;  
3 int b = a;  
4 b += 2;  
5 Console.WriteLine("a=" + a + " y b=" + b + ".");
```

Ejemplo 1:

```
2 int a = 3;  
3 int b = a;  
4 b += 2;  
5 Console.WriteLine("a=" + a + " y b=" + b + ".");
```

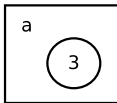


(a) `int a = 3;`

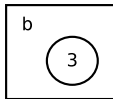
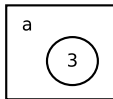
Tipos por valor

Ejemplo 1:

```
2 int a = 3;  
3 int b = a;  
4 b += 2;  
5 Console.WriteLine("a=" + a + " y b=" + b + ".");
```



(a) `int a = 3;`

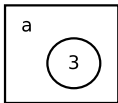


(b) `int b = a;`

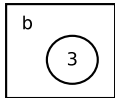
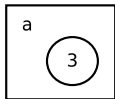
Tipos por valor

Ejemplo 1:

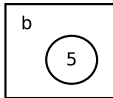
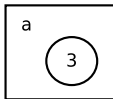
```
2 int a = 3;  
3 int b = a;  
4 b += 2;  
5 Console.WriteLine("a=" + a + " y b=" + b + ".");
```



(a) `int a = 3;`



(b) `int b = a;`



(c) `b += 2`

Ejemplo 2:

```
2 int a = 3;  
3 f(a);  
4 Console.WriteLine(a);  
5  
6 void f(int b) {  
7     b += 2;  
8 }
```

¿Qué imprime este programa?

Ejemplo 2:

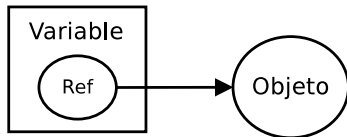
```
2 int a = 3;  
3 f(a);  
4 Console.WriteLine(a);  
5  
6 void f(int b) {  
7     b += 2;  
8 }
```

¿Qué imprime este programa?

Observación: Al dar “a” como parámetro implícitamente se hace un $b = a$.

Tipos por referencia

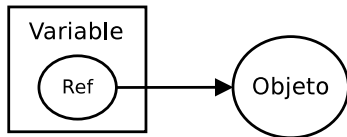
Por referencia: La variable *contiene* una referencia al objeto.



Ejemplos: string, arreglos y clases.

Tipos por referencia

Por referencia: La variable *contiene* una referencia al objeto.



Ejemplos: string, arreglos y clases.

Consecuencia: $b = a$, copia la referencia al objeto a en b .

Tipos por referencia

```
2 class Persona {  
3     private string n;  
4     public Persona(string n) {  
5         this.n = n;  
6     }  
7     public void CambiarNombre(string nombre) {  
8         n = nombre;  
9     }  
10    public void MostrarNombre() {  
11        Console.WriteLine(n);  
12    }  
13 }
```

Tipos por referencia

Ejemplo 1:

```
2 Persona juan = new Persona("juan");  
3 Persona pedro = juan;  
4 pedro.CambiarNombre("pedro");  
5 juan.MostrarNombre();
```

¿Qué muestra `juan.MostrarNombre()`?

Tipos por referencia

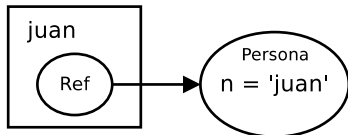
Ejemplo 1:

```
2 Persona juan = new Persona("juan");  
3 Persona pedro = juan;  
4 pedro.CambiarNombre("pedro");  
5 juan.MostrarNombre();
```

¿Qué muestra juan.MostrarNombre()?

Ejecución del código:

```
Persona juan = new Persona("juan")
```



Tipos por referencia

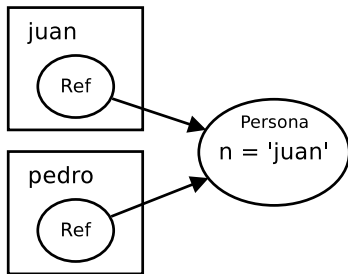
Ejemplo 1:

```
2 Persona juan = new Persona("juan");  
3 Persona pedro = juan;  
4 pedro.CambiarNombre("pedro");  
5 juan.MostrarNombre();
```

¿Qué muestra `juan.MostrarNombre()`?

Ejecución del código:

`Persona pedro = juan`



Tipos por referencia

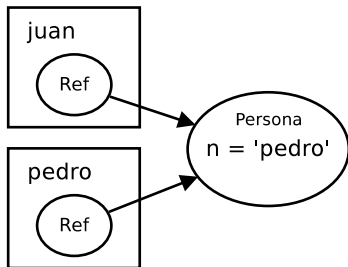
Ejemplo 1:

```
2 Persona juan = new Persona("juan");  
3 Persona pedro = juan;  
4 pedro.CambiarNombre("pedro");  
5 juan.MostrarNombre();
```

¿Qué muestra `juan.MostrarNombre()`?

Ejecución del código:

`pedro.CambiarNombre("pedro")`



Tipos por referencia

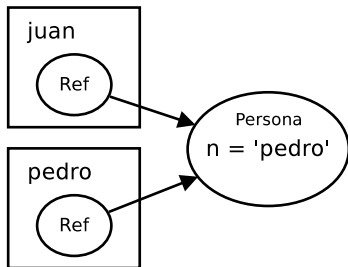
Ejemplo 1:

```
2 Persona juan = new Persona("juan");  
3 Persona pedro = juan;  
4 pedro.CambiarNombre("pedro");  
5 juan.MostrarNombre();
```

¿Qué muestra `juan.MostrarNombre()`?

Ejecución del código:

```
pedro.CambiarNombre("pedro")
```



Por este motivo, `juan.MostrarNombre()` muestra "pedro".

Tipos por referencia

Ejemplo 2:

```
2 Persona juan = new Persona("juan");
3 YoTeBautizoPedro(juan);
4 juan.MostrarNombre();
5
6 void YoTeBautizoPedro(Persona b) {
7     b.CambiarNombre("Pedro");
8 }
```

¿Qué imprime `juan.MostrarNombre()`?

Tipos por referencia

Ejemplo 2:

```
2 Persona juan = new Persona("juan");
3 YoTeBautizoPedro(juan);
4 juan.MostrarNombre();
5
6 void YoTeBautizoPedro(Persona b) {
7     b.CambiarNombre("Pedro");
8 }
```

¿Qué imprime `juan.MostrarNombre()`?

Observación: Al dar `juan` como parámetro implícitamente se hace un `b = juan`.

Output Arguments

The out keyword

También existe el keyword `out` que fuerza a nuestros argumentos a entregarse como referencia es vez de valor.

```
2 public class OutClass
3 {
4     public static void IsEven(int number, out bool result)
5         => result = (number % 2 == 0);
6 }
```

The out keyword

También existe el keyword `out` que fuerza a nuestros argumentos a entregarse como referencia es vez de valor.

```
2 public class OutClass
3 {
4     public static void IsEven(int number, out bool result)
5         => result = (number % 2 == 0);
6 }
```

`out` nos permite sobreescibir un valor desde el argumento.

```
2 bool isEven = true;
3 OutClass.IsEven(5, out isEven);
4 Console.WriteLine(isEven); //False
```

Output Arguments

Técnicamente, este método también tiene un output argument:

```
2 public static void EliminateDuplicationFrom(List<int> collection)
```

Pero tiene un nombre que hace obvio para el lector que está pasando

Output Arguments

Técnicamente, este método también tiene un output argument:

```
2 public static void EliminateDuplicationFrom(List<int> collection)
```

Pero tiene un nombre que hace obvio para el lector que está pasando

Para tu entrega:

- Evita utilizar la keyword out
- Usa funciones con muy buenos nombres si te ves obligado a modificar un tipo por referencia

Estructuras

Estructuras

Las estructuras son clases que se fuerzan a ser de **tipo por valor**.

Estructuras

Las estructuras son clases que se fuerzan a ser de **tipo por valor**.

```
2 class CPoint {  
3     private int _x;  
4     private int _y;  
5     public CPoint(int x, int y) {  
6         _x = x;  
7         _y = y;  
8     }  
9     public void Square() {  
10        _x = _x * _x;  
11        _y = _y * _y;  
12    }  
13    public void Print() { Console.WriteLine($"({_x},{_y})"); }  
14 }
```

Estructuras

Las estructuras son clases que se fuerzan a ser de **tipo por valor**.

```
2 struct SPoint {
3     private int _x;
4     private int _y;
5     public SPoint(int x, int y) {
6         _x = x;
7         _y = y;
8     }
9     public void Square() {
10         _x = _x * _x;
11         _y = _y * _y;
12     }
13     public void Print() { Console.WriteLine($"({_x},{_y})"); }
14 }
```

La única diferencia entre `SPoint` y `CPoint` es que `SPoint` es un `struct` y `CPoint` es una `class`. Pero esto hace que sus comportamientos sean distintos:

La única diferencia entre SPoint y CPoint es que SPoint es un struct y CPoint es una class. Pero esto hace que sus comportamientos sean distintos:

1) Las comparaciones entre structs son a nivel de sus atributos.

```
2 SPoint sp1 = new SPoint(3,4);  
3 SPoint sp2 = new SPoint(3,4);  
4 Console.WriteLine(sp1.Equals(sp2)); // True  
5  
6 CPoint cp1 = new CPoint(3,4);  
7 CPoint cp2 = new CPoint(3,4);  
8 Console.WriteLine(cp1.Equals(cp2)); // False
```

La única diferencia entre SPoint y CPoint es que SPoint es un struct y CPoint es una class. Pero esto hace que sus comportamientos sean distintos:

2) Las structs son tipos por valor por lo que asignaciones crean copias del struct.

```
2 SPoint sp1 = new SPoint(3,4);
3 SPoint sp2 = sp1;
4 sp1.Square();
5 sp1.Print(); // (9,16)
6 sp2.Print(); // (3,4)
7
8 CPoint cp1 = new CPoint(3,4);
9 CPoint cp2 = cp1;
10 cp1.Square();
11 cp1.Print(); // (9,16)
12 cp2.Print(); // (9,16)
```

Para que todo esto funcione – que podamos definir *clases por valor* – los struct deben cumplir un par de condiciones:

Para que todo esto funcione – que podamos definir *clases por valor* – los struct deben cumplir un par de condiciones:

- Al igual que las clases, los estructuras pueden tener atributos y métodos.
- Al momento de ser creadas, **todos** sus atributos se les deben asignar algún valor.

Para que todo esto funcione – que podamos definir *clases por valor* – los struct deben cumplir un par de condiciones:

- Al igual que las clases, los estructuras pueden tener atributos y métodos.
- Al momento de ser creadas, **todos** sus atributos se les deben asignar algún valor.

Normalmente, los struct se usan para crear tipos cuyo rol principal es almacenar datos (con poco o sin comportamiento asociado a ellos).